Jeron Alford, Taylor Musso, Tattimbet Zakariya

Malicious URL/Phishing Detector

Data Collection

Using the "Database of Benign and Malicious Webpages" on Kaggle, we downloaded both the training and testing datasets and put them in a folder labeled "data" in the repository. These CSV files have 12 different labels, including index, url, content, and label. Label is set to either "good", meaning a benign webpage, or "bad", meaning a malicious webpage.

We decided that we were going to use stream.jar from Homework 3 to simulate streaming, so we converted the CSV file to a JSON file to mimic the yelp.json file in Homework 3. To make it easier to populate the Bloom filter, we made a file that only contains the url and label for malicious links, omitting the benign ones so the Bloom filter will not have excess to deal with data when it is getting set up before testing.

Streaming Pipeline Implementation

As mentioned above, we are using stream.jar from Homework 3 for stream simulation. The main file, phishing_detector.py, is two main functions in its preliminary stages: main() and process_rdd(rdd). All main() does is set up the output file with the timestamp and FPR header, as well as set up the socket text stream to port 9999, call process_rdd for each line in the JSON, start the Streaming Context, and then wait for termination.

The Bloom filter algorithm will be in process_rdd(rdd) once it is finished. For right now, all process_rdd(rdd) is doing is returning if the rdd is empty, setting up the batch of urls that will go through the bit array, and then writing out the timestamp to the output file. Through testing,

we verified that the streaming pipeline works as intended and is ready to work with a Bloom filter.